

PROJEKT BIOTOP: DIE EVOLUTION EINER SIMULATION

Vom CLI-Legacy-Code zum modularen Retro-SciFi-Labor in C

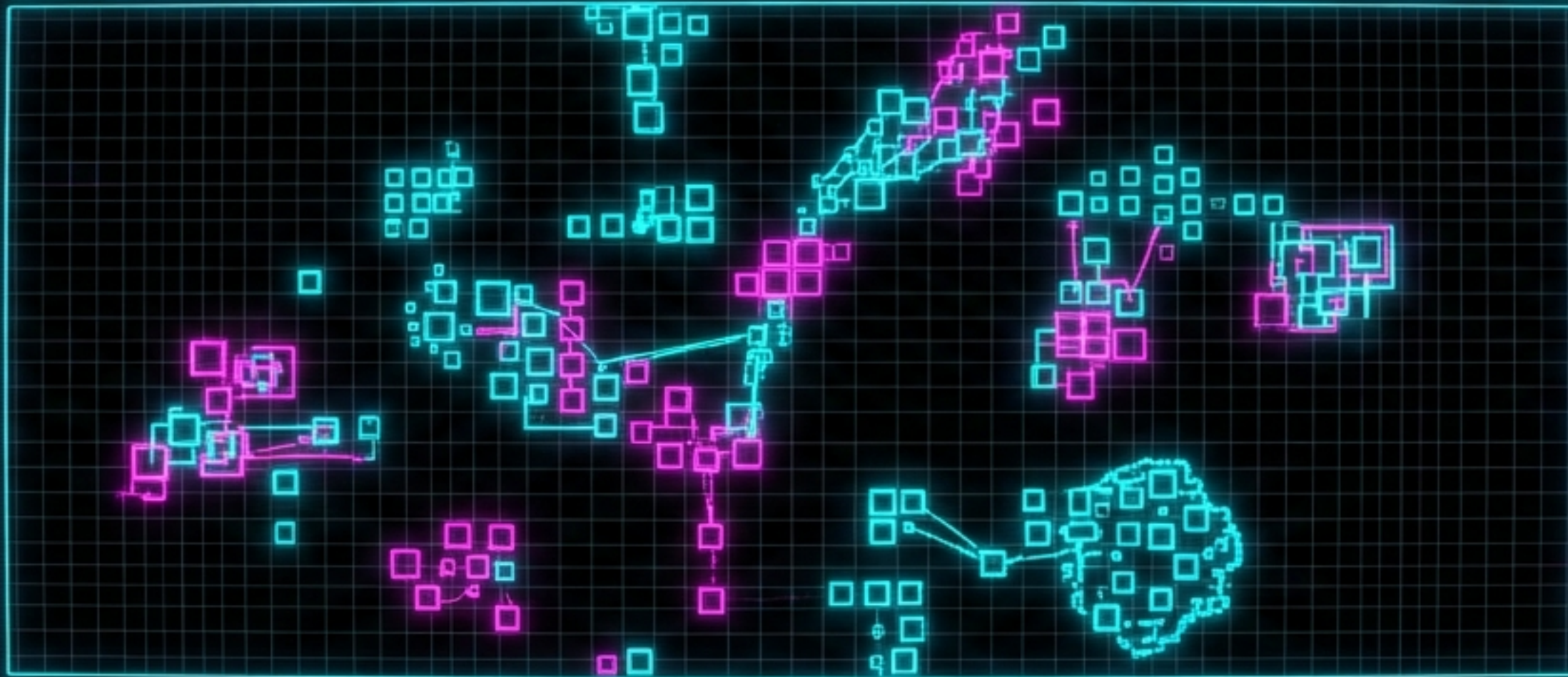
GEN: 142

POP: 400

FPS: 60

CPU: 3.2 GHz

MEM: 4096 MB



SIMULATION STATUS: ACTIVE

NETWORK: CONNECTED

MODULES: 12 LOADED

SYSTEM TIME: 14:37:22 UTC

ROLLE:
Senior Software Architekt

TECH STACK:
C, Raylib, Docker, Vibe-Coding

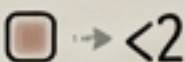
STATUS:
Release 1.0 (Stable)

DAS ZIELBILD: VOM ALGORITHMUS ZUM WETTKAMPF

DIE BASIS

Grundlage: Conway's Game of Life (1970).
Null-Personen-Spiel auf unendlichem Raster.

Regeln:

- Tod durch Einsamkeit (<2) 
- Tod durch Überbevölkerung (>3) 
- Überleben bei 2-3 Nachbarn 

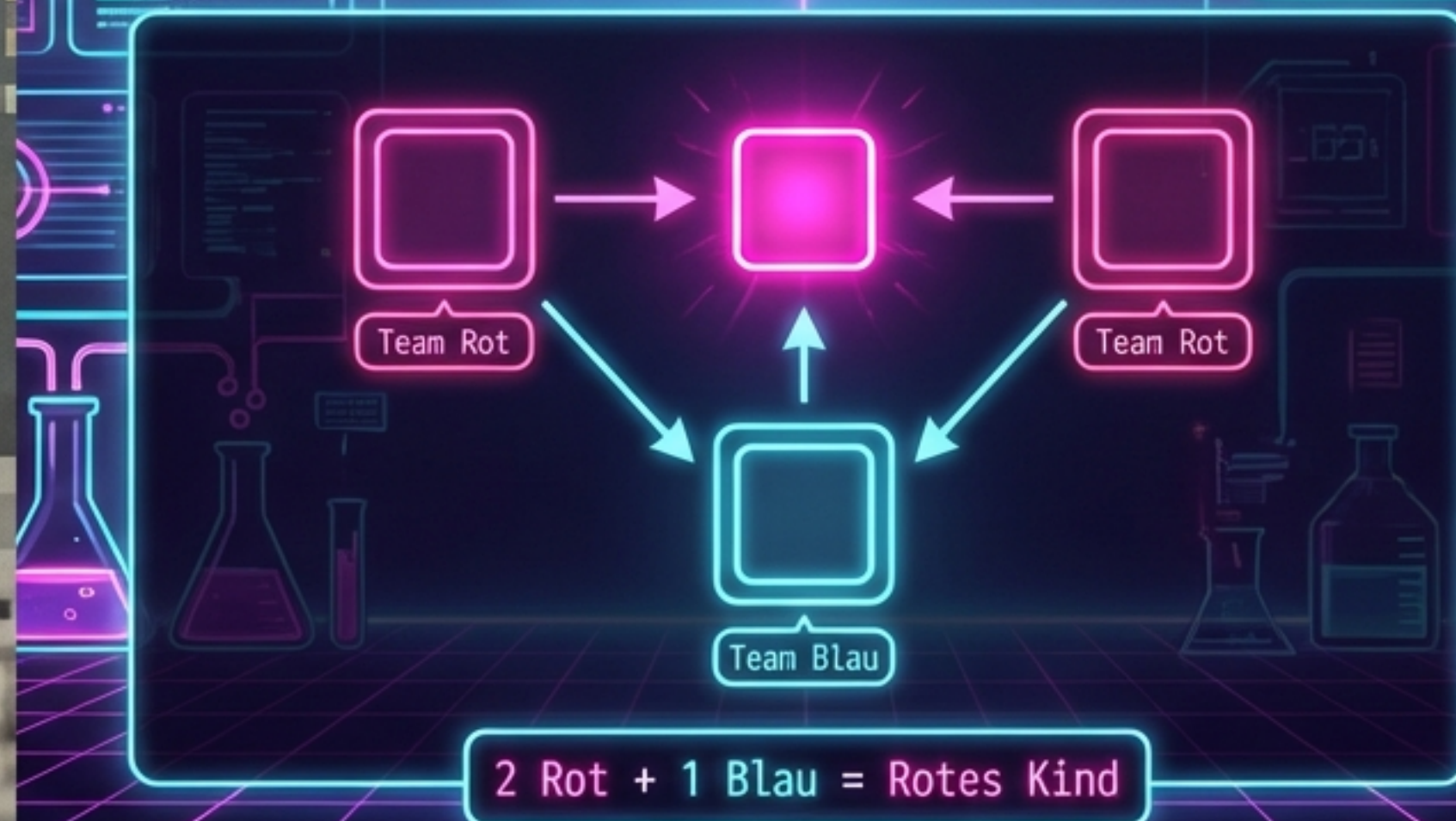
DER TWIST (BIOTOP-REGELN)

Kompetitive Komponente: Team Rot vs. Team Blau.

Geburts-Logik:

- Neue Zelle bei genau 3 Nachbarn.
- Farbe bestimmt durch Mehrheit der Eltern.

Ziel: Strategisches Vorgehen in einer Retro-Labor-Umgebung.



STATUS QUO ANTE: DER LEGACY-MONOLITH

Ausgangslage: Ein einzelnes File (main.c), rein imperativ.

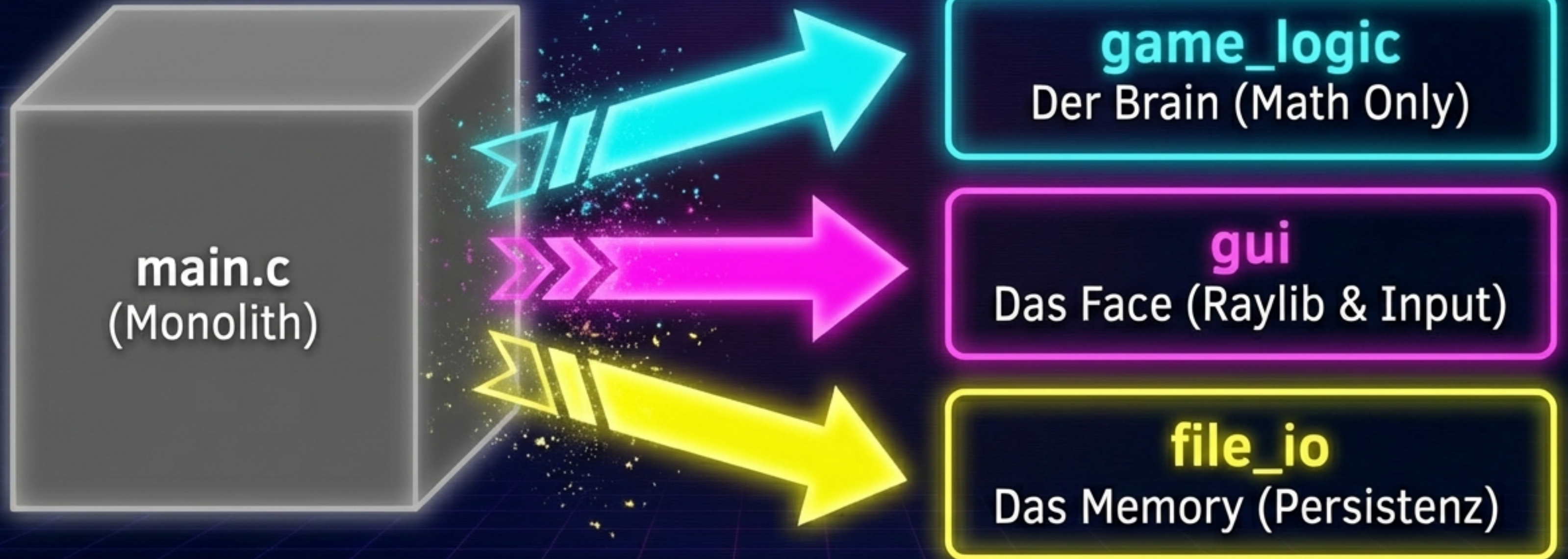
Limitierung: Manuelle Speicherverwaltung, nur CLI-Output (printf).

Code Smell: Hardcodierte Randfall-Prüfung für jede Ecke und Kante.

```
// Sonderfall: i = Eckzelle oben, links
if (i == 0) {
    zellen = current->grid[i + (cols * rows) - 1] + ...
}
else if (i == (cols - 1)) {
    // ... redundante Logik für Ecke rechts oben
}
else if (i == (cols * rows - 1)) {
    // ... noch mehr if-else Kaskaden
}
```

**FUNKTIONAL, ABER
NICHT SKALIERBAR**

PHASE I: ARCHITEKTONISCHE DEKONSTRUKTION



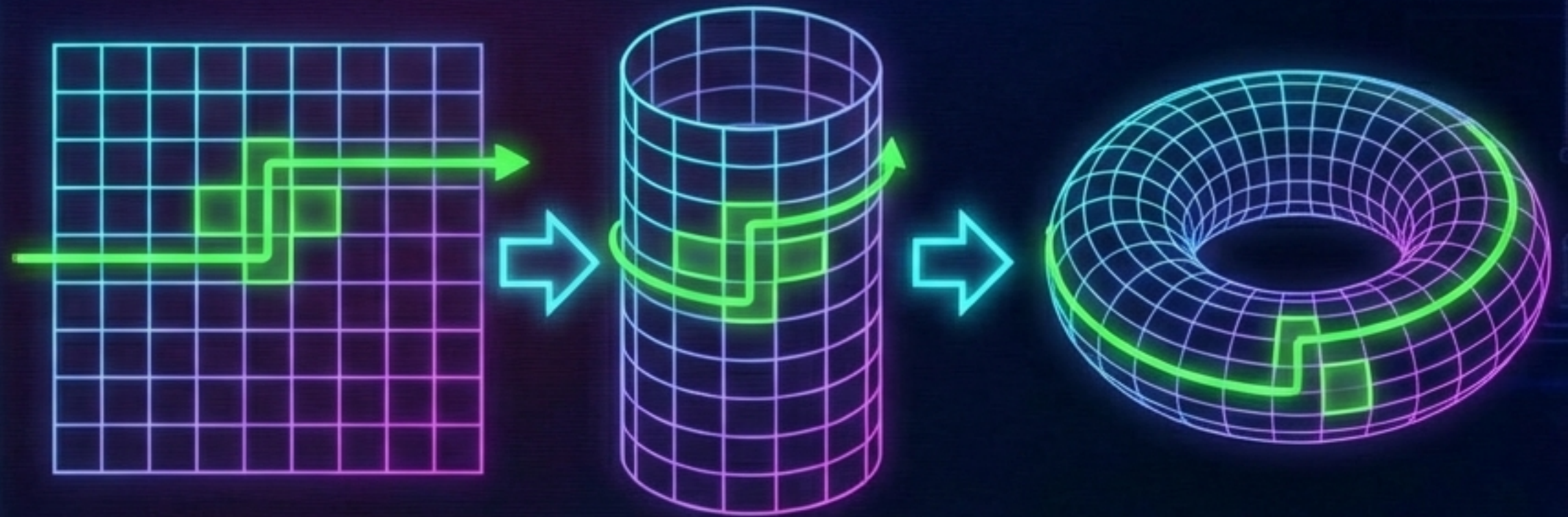
*Separation of Concerns: Die Logik weiß nichts von Pixeln.
Die GUI weiß nichts von Dateisystemen.*

TOPOLOGIE-LÖSUNG: DER TORUS

Problem: Endliches Array, aber unendliche Spiel-Illusion.

Lösung: Modulo-Arithmetik für Weltumrundung.

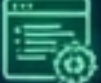
Formel:
 $(x + \text{width}) \% \text{width}$



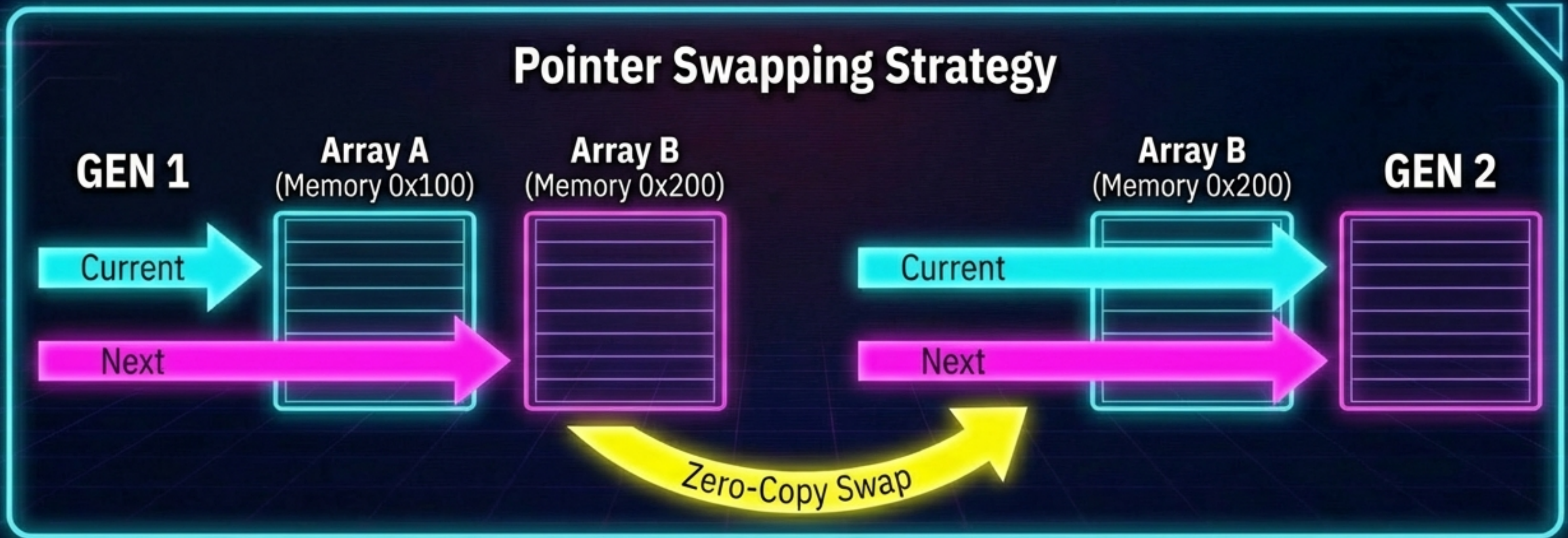
Vorher: 150 Zeilen
If-Else für Ränder

Nachher: 1 generische Funktion
`count_neighbors_by_team()`

C-LEVEL PERFORMANCE & SPEICHER-OPTIMIERUNG

 **Datenstruktur:** 1D Integer Array für maximale Cache Locality (0=Tot, 1=Rot, 2=Blau).

 **Inlining:** Makros (COUNT_NEIGHBOR) statt Funktionsaufrufe in der Hot Loop.



DIE GUI-REVOLUTION: INTEGRATION VON RAYLIB

- **Entscheidung:** Raylib für OpenGL-Beschleunigung und C-Kompatibilität.
- **Architektur:** Finite State Machine (FSM) steuert den Programmfluss.



UX-DESIGN: DAS DIGITALE BIOLABOR

- **Atmosphäre:** Modern Retro, Dark Mode (#1a1a1a), Neon-Akzente.
- **Wording:** 'Eco-Bloom Cycle' statt 'Round'. Der Nutzer ist Forscher.
- **Rendering:** Padding-Technik statt Linien-Zeichnen für performantes Raster.

CYCLES: 0042

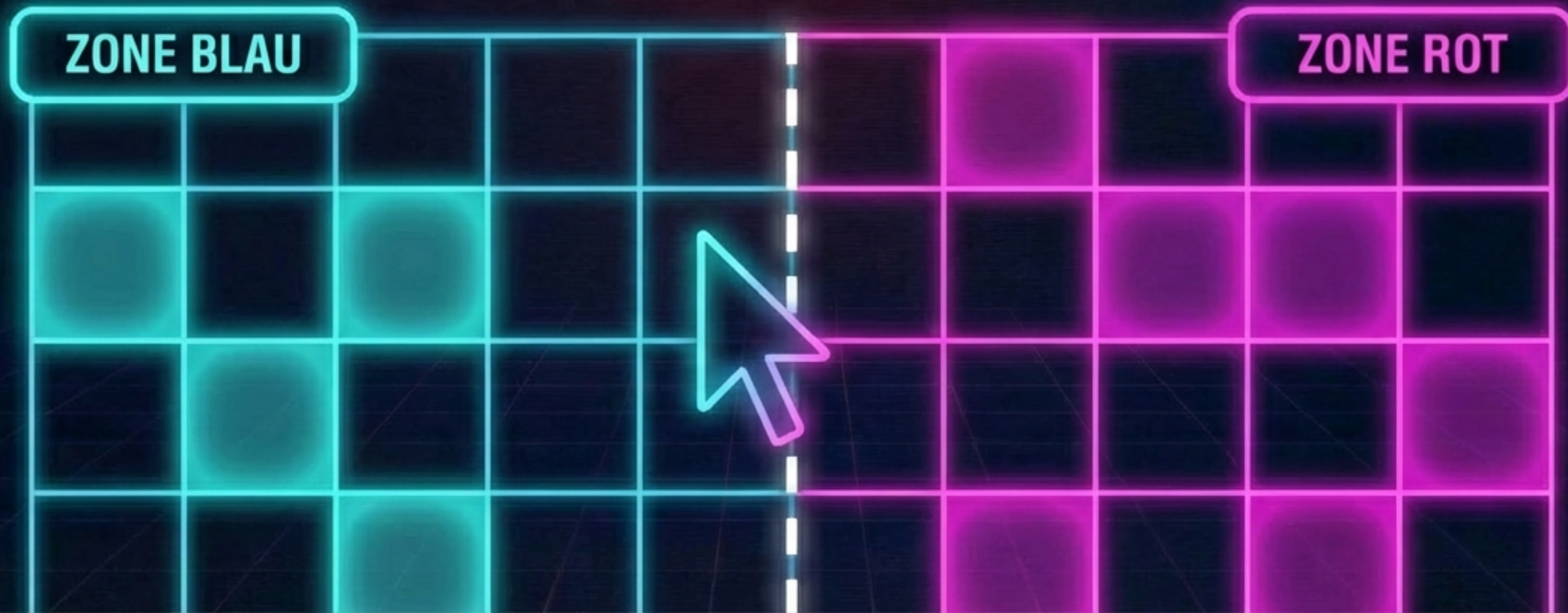
TEAM RED: 150 | TEAM BLUE: 145

[SPACE] PAUSE [S] SAVE



INTERAKTIVER EDITOR & HEMISPHERÄN

- **Fairness durch Geometrie:** Das Spielfeld ist im Editor geteilt.
- **Team Blau:** Linke Hemisphäre. **Team Rot:** Rechte Hemisphäre.
- **Logik:** Coordinate-Mapping in gui.c validiert Input gegen $(\text{cols} / 2)$.



DATEN-PERSISTENZ & PROTOKOLL-SYSTEM V2

- **Format:** Custom **.bio** Files (Text-basiert, kein JSON-Overhead).
- **Safety:** 'Save-on-Start' sichert Zustand vor Simulationsbeginn.
- **Appending:** Ergebnisse werden als Footer an die Quelldatei angehängt.

experiment_01.bio

Biotop-Config-v2

Grid Data: 0010200...

RESULTS: Winner=RED,
Cycles=500

VERIFIZIERBARKEIT: DAS IN-APP REPLAY-SYSTEM

- **Ziel:** Reproduzierbares Forschungswerkzeug.
- **Tech:** Nutzung von dirent.h zum Scannen von 'biotope_results/'.

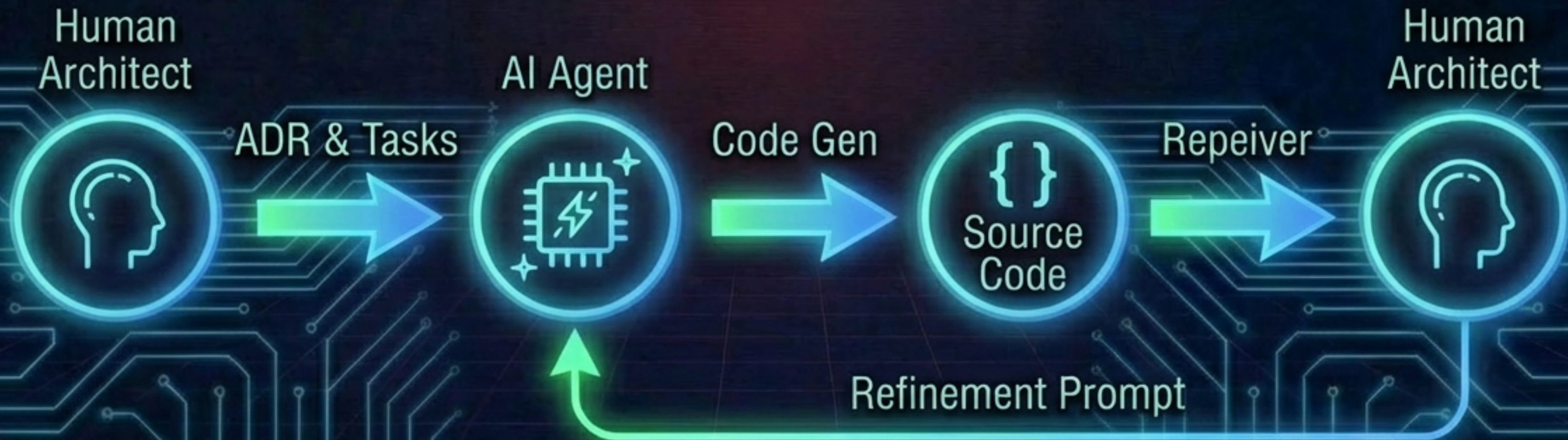
FILE BROWSER

2023-10-27_Test_Run_A.bio
2023-10-27_Alpha_Sim.bio
2023-10-28_Tournament_Final.bio

LOAD SIMULATION

METHODIK: DER VIBE-CODING PROZESS

- **Rollenverteilung:**
- **Mensch** = Senior Architekt & Reviewer.
- **AI (LLM)** = Junior Developer & Implementierer.



HERAUSFORDERUNGEN & DEBUGGING



Torus Y-Achsen Fehler: Negatives Wrapping führte zu Speicherzugriffsfehlern.
Fix: ``i - (cols * (rows - 1))``



Keyboard Layouts: Raylib ``KEY_PLUS`` inkompatibel mit DE-Tastaturen.
Fix: Fallback auf ``KEY_EQUAL`` und ``PageUp/Down``.



AI-Halluzinationen: Placeholder wie ``...`` im Code verursachten Compile-Errors.
Fix: Manuelle Code-Reviews und strictes Prompting.



UI-Scaling: Textüberlappungen im HUD.
Fix: Dynamische Breitenberechnung mit ``MeasureText``.

STATUS QUO & SYSTEM-QUALITÄT



Funktionalität.

Config, Editor, Sim,
Save/Load, Replay.



Stabilität.

Keine Speicherlecks
(Valgrind verified).
DoS-Schutz (Max Grid 2k).



Code-Qualität.

Modular, C-Standard kon-
form, ADR-dokumentiert.



Ergebnis.

Ein präzises, robustes und
ästhetisches System.



FAZIT: KOMPLEXITÄT DURCH ARCHITEKTUR BEHERRSCHEN

FAZIT: KOMPLEXITÄT DURCH ARCHITEKTUR BEHERRSCHEN

**Projekt Biotop beweist: Legacy-Code kann durch Refactoring
und Vibe-Coding zu neuem Leben erwachen.**

Quellcode & Demo verfügbar am Messestand.